

TRIBHUWAN UNIVERSITY
INSTITUTE OF ENGINEERING
KATHMANDU ENGINEERING COLLEGE

KALIMATI, KATHMANDU



Minor Project Mid Defense Report

On

Tribal: Find Your Tribe

[Code Number: ENCT354]

BY

Sampada Dahal	[KAT080BCT067]
Selina Pageni	[KAT080BCT073]
Shlesha Aryal	[KAT080BCT075]
Sukriti Dhakal	[KAT080BCT083]

TO

DEPARTMENT OF COMPUTER ENGINEERING

KATHMANDU, NEPAL

June 2026

Acknowledgement

We would like to express our sincere gratitude to **Mr. Sudeep Shakya**, Head of the Department of Computer Engineering at Kathmandu Engineering College, for his leadership and support. We are also highly grateful to our Year Coordinator, **Mr. Nabin Neupane**, and our Project Coordinator, **Ms. Krista Byanju**, for their invaluable guidance, administrative support, and continuous encouragement during the formulation of this minor project proposal.

Furthermore, we extend our thanks to our project supervisor and the faculty members of the Department of Computer Engineering for their constructive feedback and expertise. Finally, we are thankful to our peers and colleagues who provided valuable suggestions that helped refine our project design and methodologies.

Abstract

Finding compatible roommates and meeting like-minded people in a new city is a problem that many people face today. Most of the existing platforms either focus only on roommate searching or only on social activities. There is no single platform that can help users find roommates, join activities, and build a trusted community all at once. TRIBAL is a mobile application built using Flutter that tries to solve this problem. The application allows users to find compatible roommates based on their lifestyle preferences, find activity partners, create and join local events, and chat with other users. The platform also includes a compatibility scoring algorithm that matches users based on factors like budget, sleep schedule, cleanliness habits, pet preferences, and other lifestyle-related questions collected during onboarding. The backend of the application is built using Django and uses PostgreSQL as the database. Authentication is handled using JWT tokens and real-time chat is being implemented using Django Channels with WebSockets.

Keywords: Roommate Matching, Compatibility Scoring, Activity Partner Discovery, Mobile Application, Flutter, Django, PostgreSQL, Real-Time Chat, Community Building, Lifestyle Preferences

Table of Contents

Acknowledgement	ii
Abstract	iii
Table of Contents	iv
List of Figures	vi
List of Tables	vii
Abbreviations	viii
1 Introduction	1
1.1 Background Theory	1
1.2 Problem Statements	1
1.3 Objectives	2
1.4 Scope and Applications	2
1.5 Literature Review	4
1.5.1 Foundations of Recommendation and Matching Systems	4
1.5.2 Roommate Matching and Preference Modeling	4
1.5.3 Experimental Studies on Roommate Matching	4
1.5.4 Event-Based Social Networking	5
1.5.5 Review of Existing Solutions	5
1.5.6 Research Gap	5
2 Methodology	7
2.1 Process Model	7
2.2 System Block Diagram	9
2.3 Algorithm	10
2.3.1 Roommate Compatibility Scoring Algorithm	10
2.3.2 Deal Breakers	11
2.3.3 Algorithm Steps	14
2.3.4 Final Score Calculation	15

2.4	Flowchart	16
2.5	UML Diagrams	17
2.5.1	Class Diagram	17
2.5.2	Sequence Diagram	19
2.5.3	User Flow Diagram	20
2.6	Tools Used	21
3	Epilogue	22
3.1	Tasks Completed	22
3.2	Remaining Tasks	22
3.3	Updated Gantt Chart	23
	References and Bibliography	24
	Screenshots	25

List of Figures

2.1	Agile Process Model	8
2.2	System Block Diagram of TRIBAL	9
2.3	System Flowchart	16
2.4	Class Diagram	17
2.5	Sequence Diagram	19
2.6	User Flow Diagram	20

List of Tables

2.1	Compatibility Scoring Weights	10
2.2	Deal Breaker Rules	11
2.3	Tools and Technologies Used	21

Abbreviations

API	Application Programming Interface
Dart	Dart Programming Language
Django	Django Web Framework
Figma	Figma Design Tool
Flutter	Flutter UI Framework
Git	Git Version Control System
GitHub	GitHub Code Hosting Platform
JWT	JSON Web Token
MVC	Model-View-Controller
PostgreSQL	PostgreSQL Database
Postman	Postman API Testing Tool
Python	Python Programming Language
REST	Representational State Transfer
UI	User Interface
UML	Unified Modeling Language
UX	User Experience
VS Code	Visual Studio Code

Chapter 1 Introduction

1.1 Background Theory

In recent years, the increasing number of students and working professionals moving to new cities has created a growing need for platforms that help users find compatible roommates and build social connections. Existing social media and roommate-finding applications either lack compatibility-based matching or focus only on basic profiles, leaving a gap in the market.

TRIBAL is a cross-platform mobile application developed using Flutter and Django to address this problem. It enables users to create profiles, answer lifestyle-based onboarding questions, and find compatible roommates and activity partners through a weighted compatibility scoring algorithm. The application also supports event creation, participation, and real-time chat between matched users.

The weighted compatibility scoring approach was chosen over machine learning because it is transparent, easier to implement within the scope of a minor project, and provides meaningful matches based on users' preferences and interests.

1.2 Problem Statements

People who want to find others with shared interests often have no proper platform to do so. General social media platforms are too broad and not designed for this kind of need. Activity-specific platforms like Meetup are not widely used in Nepal and do not support roommate finding at all.

The main problems that TRIBAL tries to address are:

- Finding compatible roommates and building a social circle is challenging for students and young professionals who move to new cities, as they often rely on informal methods such as friends or social media groups.
- Existing platforms do not provide a single solution for roommate matching, activity partner discovery, and community building, while also overlooking important lifestyle compatibility factors.

- TRIBAL addresses these challenges by offering a unified platform for roommate matching based on compatibility, activity partner discovery, event participation, and secure in-app chat.

TRIBAL aims to solve these problems by providing a single mobile platform where users can find compatible roommates, discover activity partners, join events, and chat with matches in a safe environment.

1.3 Objectives

The main objectives of this project are:

- To develop a mobile platform that helps users find compatible roommates by evaluating lifestyle preferences through a compatibility scoring system.
- To provide a platform where users can create, browse, and join local events to encourage community engagement.
- To facilitate secure one-to-one communication between matched users through an in-app real-time chat system.

1.4 Scope and Applications

The scope of this project as a minor project includes the following features:

- Develop a cross-platform Flutter application that enables users to find compatible roommates and activity partners through secure authentication, profile management, and preference-based matching.
- Provide essential social features, including real-time chat, event creation and participation, and interest-based activity discovery to help users build meaningful connections.
- Address the challenges of finding roommates and social networks, particularly for students and newcomers in Nepal, while serving as a scalable prototype for future enhancements such as verified profiles and location-based recommendations.

For the purpose of this minor project, the focus is on completing the core features including the compatibility matching system, real-time chat, and event module. The

application is not intended for commercial deployment at this stage, but it demonstrates a working prototype of the complete system.

1.5 Literature Review

1.5.1 Foundations of Recommendation and Matching Systems

Recommendation and matching systems have become increasingly important in helping users identify suitable people, resources, or activities based on their preferences. In social networking environments, recommendation techniques are commonly used to reduce information overload by analyzing user interests, behaviors, and contextual information. Event recommendation systems further extend these techniques by considering factors such as user preferences, social relationships, and event characteristics to recommend suitable activities. Liao *et al.* proposed an intelligent group event recommendation system that combines user interests with social network information to improve the quality of event recommendations for groups rather than individuals [1].

1.5.2 Roommate Matching and Preference Modeling

Roommate matching is a one-sided matching problem in which individuals are paired based on mutual compatibility rather than belonging to two separate groups. Traditional roommate matching approaches primarily focus on matching individuals according to their preferences. However, the quality of accommodation also plays a significant role in user satisfaction.

Hosseini *et al.* extended the traditional roommate matching problem by jointly considering both roommate assignments and room allocations. Their work introduces strategyproof mechanisms that ensure users cannot obtain a better outcome by misrepresenting their preferences. The study highlights that both roommate compatibility and room quality contribute to an effective matching outcome and should therefore be considered together during the allocation process [2].

1.5.3 Experimental Studies on Roommate Matching

While many roommate matching algorithms have been studied theoretically, fewer studies investigate how such mechanisms perform in practice. Molis and Veszteg conducted experimental research on the roommate problem by observing participant behavior during decentralized matching processes. Their findings demonstrate that participants generally attempt to improve their current matches through mutually beneficial exchanges until a

stable configuration is reached. The study provides valuable insights into how theoretical matching models behave under realistic conditions and emphasizes the importance of user preferences in achieving stable roommate assignments [3].

1.5.4 Event-Based Social Networking

Beyond roommate allocation, modern student communities benefit from platforms that encourage social interaction and participation in events. Event recommendation systems help users discover activities that match their interests while also strengthening community engagement.

The intelligent recommendation framework proposed by Liao *et al.* incorporates both user preferences and social relationships when recommending group events. By considering multiple users simultaneously, the system produces recommendations that better reflect collective interests than traditional individual recommendation methods. This demonstrates the potential of integrating recommendation techniques into social networking platforms designed for students and shared communities [1].

1.5.5 Review of Existing Solutions

Existing roommate matching platforms primarily focus on helping users find accommodation or compatible roommates. Although these systems support profile matching and preference filtering, they generally do not provide integrated features that promote long-term social interaction after users have been matched.

Similarly, event recommendation platforms concentrate on suggesting activities based on user interests but do not address roommate compatibility or housing allocation. The literature therefore indicates that roommate matching and event recommendation have largely been studied as separate research problems, with limited integration between the two domains [1], [2], [3].

1.5.6 Research Gap

The reviewed literature demonstrates significant progress in both roommate matching and group event recommendation. Hosseini *et al.* address the challenge of jointly matching roommates and rooms through strategyproof mechanisms, while Liao *et al.* improve

social engagement using intelligent group event recommendation techniques. Molis and Veszteg provide experimental evidence supporting the practical behavior of roommate matching systems.

However, existing studies do not present a unified platform that combines roommate compatibility, accommodation preferences, social networking, event recommendations, and real-time communication within a single system. This gap motivates the development of **TRIBAL**, which integrates roommate matching, community interaction, event participation, and communication features into one application to improve both residential compatibility and social engagement among students.

Chapter 2 Methodology

2.1 Process Model

The team is following the Agile process model for the development of TRIBAL. Agile was chosen because it allows the project to be broken down into small, manageable parts called sprints, and each sprint produces a working piece of the application. This is useful for a minor project because requirements can be refined as the team learns more, and progress can be shown at regular intervals like this mid-term defense. The project has been divided into the following sprints:

- Sprint 1: Project setup, requirement analysis, and UI/UX wireframing in Figma.
- Sprint 2: Flutter onboarding flow, authentication screens (Login and Signup), and post-signup profile completion screens.
- Sprint 3: Django backend setup, PostgreSQL database design, and JWT-based authentication APIs.
- Sprint 4: Design and implementation of the roommate compatibility scoring algorithm (current sprint).
- Sprint 5: Real-time chat system using Django Channels and WebSockets.
- Sprint 6: Event creation and joining module.
- Sprint 7: Integration testing between the Flutter frontend and Django backend.
- Sprint 8: Final testing, bug fixing, and documentation.

At the time of this report, the team has completed Sprints 1 to 4 and has started work on Sprint 5.

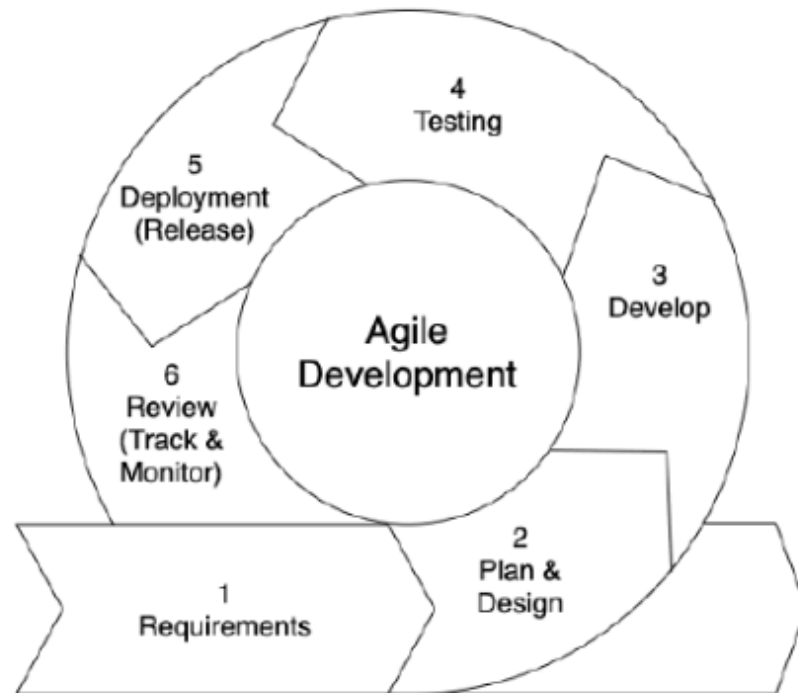


Figure 2.1: Agile Process Model

Figure 2.1 shows the Agile development cycle that the team is following for this project. The diagram is a circular flow with six stages: Requirements, Plan & Design, Develop, Testing, Deployment (Release), and Review (Track & Monitor), all revolving around a central label that says “Agile Development”. The circular arrangement shows that after each review cycle, the team loops back to plan the next set of requirements. This model fits our project well because we are working in short sprints, and after each sprint we review what was done and plan the next one based on what is remaining.

2.2 System Block Diagram

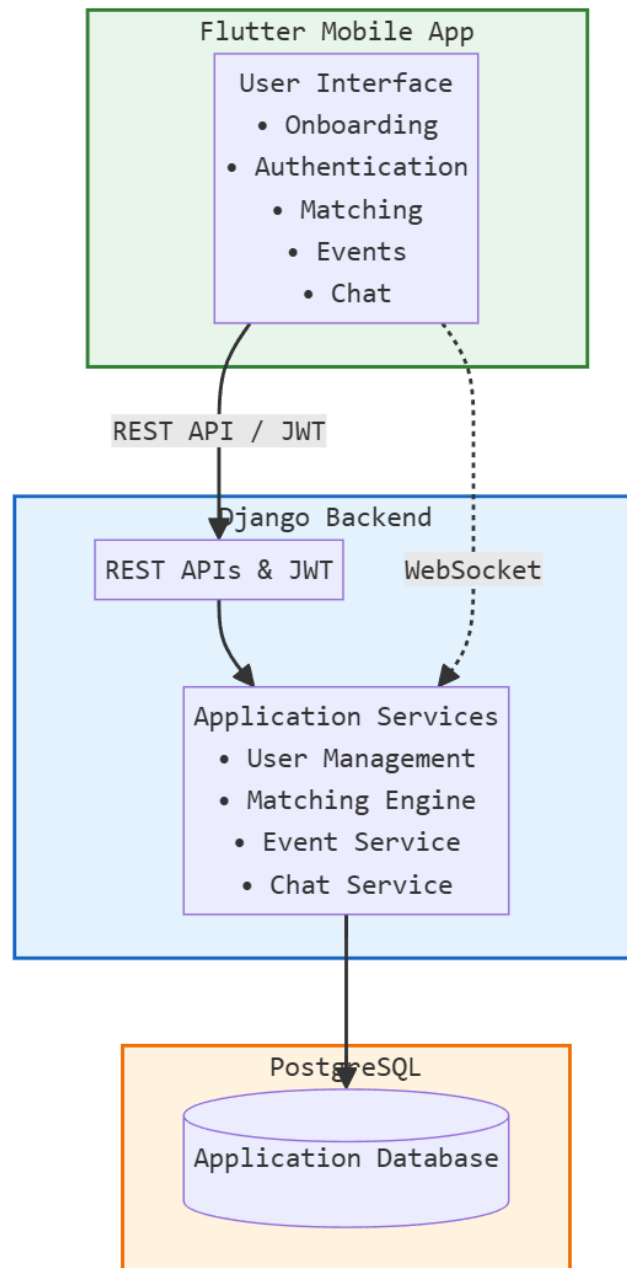


Figure 2.2: System Block Diagram of TRIBAL

Figure 2.2 shows the overall system architecture of TRIBAL divided into three layers. At the top is the Flutter Mobile App, which contains the User Interface with five sections: Onboarding, Authentication, Matching, Events, and Chat. Below it is the Django Backend, which handles two kinds of communication: regular REST API requests and JWT-based authentication on the left, and WebSocket connections for real-time chat on the right. Inside the Django layer, an Application Services block handles four services: User Management, Matching Engine, Event Service, and Chat Service. At the bottom is

PostgreSQL, represented by a database cylinder labelled Application Database. A solid arrow connects the Flutter app to the Django backend via REST API/JWT, and a dashed arrow shows the WebSocket connection used for the chat feature. This layered structure makes it easy to understand how each part of the system is responsible for a specific job.

2.3 Algorithm

2.3.1 Roommate Compatibility Scoring Algorithm

The main algorithm used in TRIBAL is the Roommate Compatibility Scoring Algorithm. It compares the onboarding responses of two users and calculates a compatibility score between 0 and 100. The algorithm is rule-based and uses weighted scoring because it is simple, transparent, and suitable for a minor project prototype. The algorithm takes two user profiles as input: the current user and a candidate roommate. It compares their lifestyle preferences such as cleanliness, budget, sleep schedule, noise level, smoking, guests, interests, study habit, food preference, and drinking habit. Input Current user profile and candidate roommate profile containing: Cleanliness preference Budget range Sleep schedule Noise preference Smoking preference Guest preference Interests Study habit Food preference Drinking preference Output A final compatibility score between 0 and 100.

Table 3.1 below shows the weight distribution used in the scoring algorithm.

Factor	Weight (%)
Cleanliness	15
Budget	15
Sleep Schedule	12
Noise Level	10
Smoking	10
Guests	8
Interests	10
Study Habit	8
Food Preference	7
Drinking	5

Table 2.1: Compatibility Scoring Weights

The general formula used to calculate the final compatibility score is:

$$\text{Final Score} = \sum (\text{Factor Match Value} \times \text{Factor Weight}) \quad (2.1)$$

Where Factor Match Value is a number between 0 and 1 that represents how closely two users' answers match for a particular factor, and Factor Weight is the percentage weight assigned to that factor as shown in Table 3.1. For factors with a small number of fixed options, such as smoking or sleep schedule, the match value is calculated using a simple rule: if both users selected the same option, the match value is 1; if their answers are close but not the same (for example, "early riser" and "no preference"), a partial match value such as 0.5 is given; and if the answers directly conflict (for example, "smoker" matched with "non-smoker, no smoking allowed"), the match value is set to 0. For factors that involve multiple selections, such as interests, the match value is calculated as the number of common interests divided by the total number of unique interests selected by both users combined. This gives a percentage overlap between the two users' interests. For numeric factors like budget, the match value decreases as the difference between the two users' stated budgets increases. If the difference is within an acceptable range, the match value stays close to 1. If the difference is very large, the match value moves closer to 0.

2.3.2 Deal Breakers

Apart from the weighted scoring, the algorithm also includes a deal breaker check. Some lifestyle differences are considered serious enough that they should not simply lower the score a little, but should instead strongly affect or override the final match. For example, if one user is a heavy smoker and the other has explicitly said they cannot live with a smoker at all, this is treated as a deal breaker rather than just a normal mismatch. Table 3.2 lists the deal breaker rules currently used in the algorithm.

Condition	Effect on Score
Smoking conflict (smoker vs. strictly non-smoking)	Score capped at a low maximum
Extreme budget mismatch (outside acceptable range)	Score capped at a low maximum
Opposite cleanliness extremes (very messy vs. very strict)	Score significantly reduced

Table 2.2: Deal Breaker Rules

When a deal breaker condition is detected, the algorithm does not let the final score go above a fixed low ceiling, even if all other factors match well. This makes sure that users are not shown as highly compatible with someone they have a fundamental lifestyle conflict with.

2.3.3 Algorithm Steps

Algorithm 1: Roommate Compatibility Scoring Algorithm

Input: Current user's profile, Candidate roommate profiles

Output: Ranked list of compatible roommates

- 1 Retrieve the current user's onboarding responses and roommate preferences;
- 2 Fetch all candidate roommate profiles from the database;
- 3 Initialize an empty list named CompatibleCandidates;
- 4 **for** *each candidate profile in the candidate list* **do**
 - 5 Set total_score $\leftarrow$ 0;
 - 6 Compute the compatibility score for each matching criterion:
 - Cleanliness preference
 - Budget preference
 - Sleep schedule
 - Noise tolerance
 - Smoking preference
 - Guest preference
 - Shared interests
 - Study habits
 - Food preference
 - Drinking preference
 - ;
 - Calculate the total compatibility score by summing all individual criterion scores;
 - Check for deal-breaker conditions (e.g., incompatible smoking preference or budget mismatch). If a deal-breaker exists, reduce the total score or apply a predefined score cap;
 - Round the final compatibility score;
 - Store the candidate profile together with its final compatibility score in CompatibleCandidates;
- 7 Sort CompatibleCandidates in descending order based on the final compatibility score;
- 8 Return the sorted list as the ranked roommate recommendations;

2.3.4 Final Score Calculation

Once all the individual factor match values are calculated and weighted, they are added together to get the final compatibility score. If a deal breaker is triggered, the score is capped according to the deal breaker rule before being returned to the user. The final score is then rounded to the nearest whole number and shown to the user as a percentage match out of 100. This score is what the user sees in the app when browsing potential roommates, helping them quickly understand how compatible they may be with someone before starting a conversation. This algorithm provides a transparent and explainable matching system. Since it does not depend on machine learning or large datasets, it is practical for the current scope of the TRIBAL minor project.

2.4 Flowchart

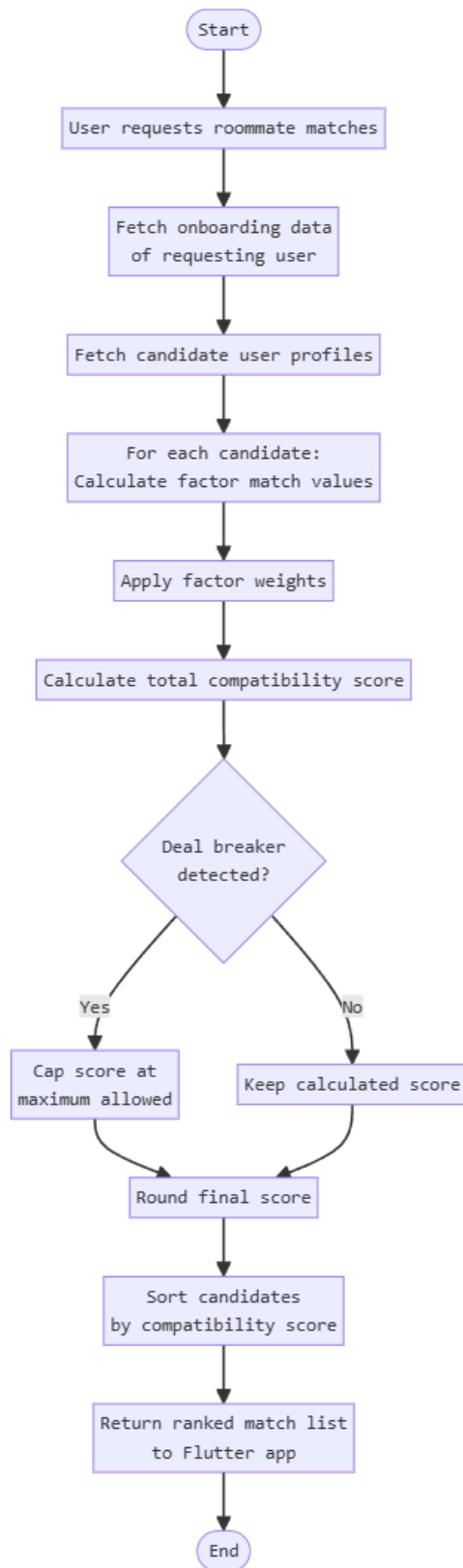


Figure 2.3: System Flowchart

Figure 2.3 shows how the roommate matching algorithm works step by step. The flowchart starts with a “Start” node, followed by the step “User requests roommate matches”. After that, the system fetches the onboarding data of the requesting user from the database, and then fetches all candidate user profiles. For each candidate, the algorithm calculates the individual factor match values and then applies the factor weights. The next step is calculating the total compatibility score, after which there is a decision diamond labelled “Deal breaker detected?”. If the answer is Yes, the score is capped at the maximum allowed value. If the answer is No, the calculated score is kept as it is. After this, the final score is rounded and all candidates are sorted by their compatibility score. Finally, the ranked list is returned to the Flutter app and the flowchart ends. This diagram helped us make sure that we had thought through all the edge cases in the algorithm before starting to code it.

2.5 UML Diagrams

2.5.1 Class Diagram

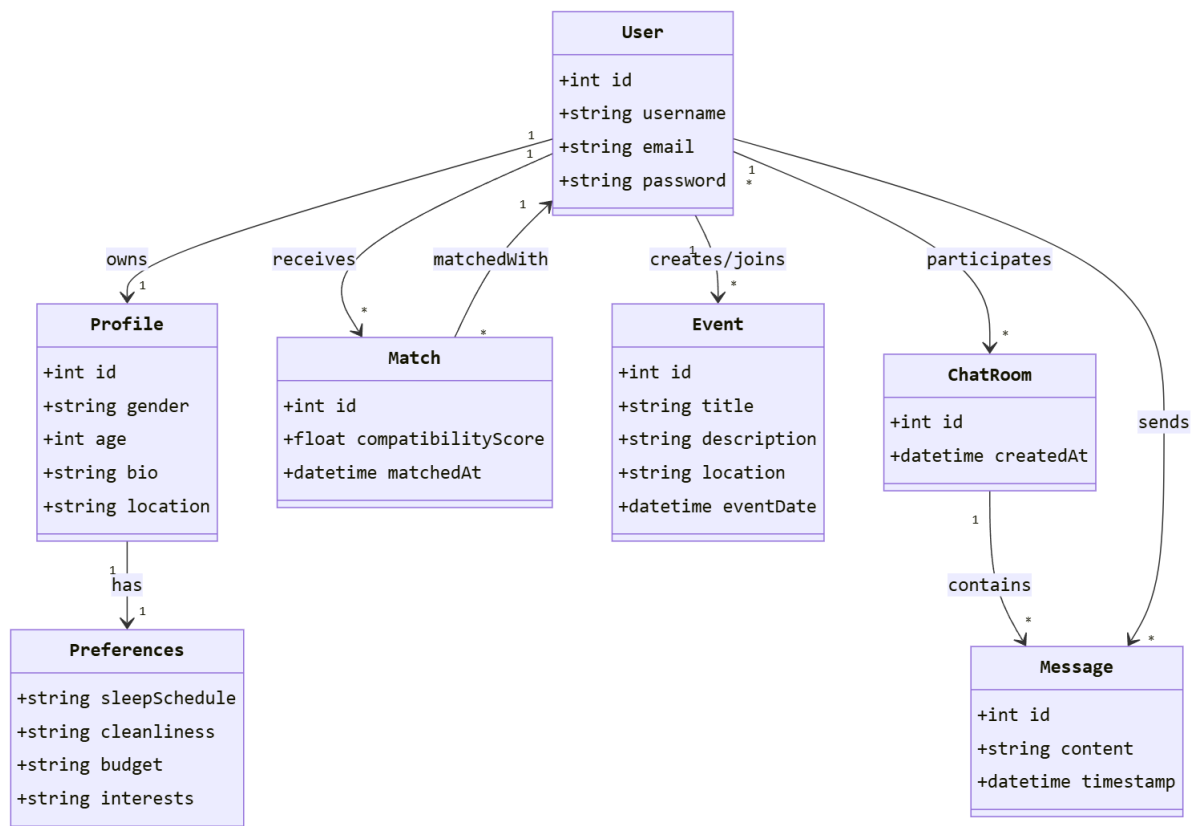


Figure 2.4: Class Diagram

Figure 2.4 shows the class diagram for the TRIBAL system. The central class is **User**, which has four attributes: `id`, `username`, `email`, and `password`. From the **User** class, five relationships branch out. The **User** owns a **Profile** (one-to-one), which holds `gender`, `age`, `bio`, and `location`. The **Profile** further has a one-to-one relationship with a **Preferences** class, which stores the lifestyle fields used in matching: `sleepSchedule`, `cleanliness`, `budget`, and `interests`. The **User** also receives **Match** objects (one-to-many), where each **Match** holds a `compatibilityScore` and a `matchedAt` timestamp. The **User** creates or joins **Event** objects, each of which has a `title`, `description`, `location`, and `eventDate`. Finally, the **User** participates in a **ChatRoom**, which contains many **Message** objects, each with a `content` string and a `timestamp`. The **User** also has a `sends` relationship to **Message** directly. This diagram shows us all the data models we need to build and how they relate to each other.

2.5.2 Sequence Diagram

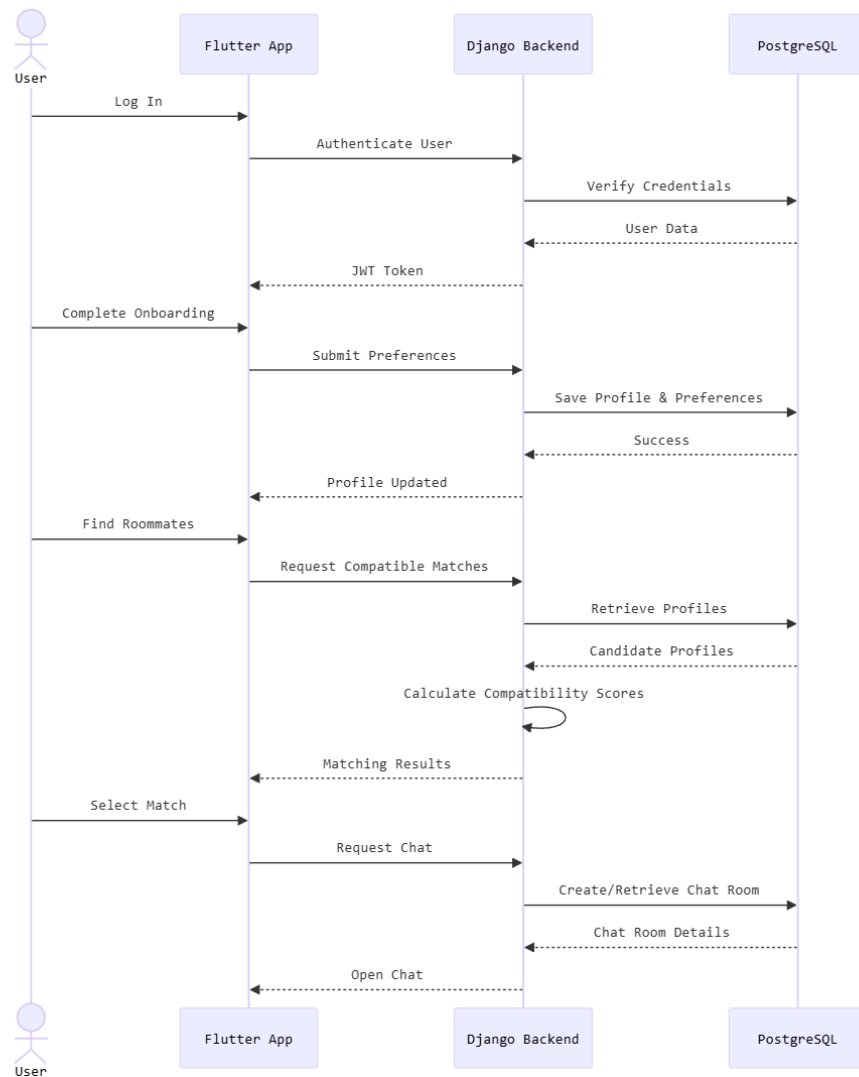


Figure 2.5: Sequence Diagram

Figure 2.5 shows the sequence of interactions between the User, the Flutter App, the Django Backend, and PostgreSQL. The diagram starts with the user logging in, after which the Flutter app sends an “Authenticate User” request to the Django backend. The backend then sends a “Verify Credentials” query to PostgreSQL, which responds with User Data. Django then returns a JWT Token to the Flutter app. In the next step, the user completes onboarding, and the Flutter app sends the preferences to Django, which saves the Profile and Preferences to PostgreSQL and confirms success. When the user taps “Find Roommates”, the Flutter app sends a “Request Compatible Matches” call to Django. Django retrieves all candidate profiles from PostgreSQL and then runs a self-call loop labelled “Calculate Compatibility Scores”. The resulting “Matching Results” are returned to the Flutter app. After the user selects a match, the Flutter app sends a

“Request Chat” to Django, which creates or retrieves a Chat Room from PostgreSQL and returns the Chat Room Details, after which the chat is opened. This diagram covers the full user journey from login to starting a conversation.

2.5.3 User Flow Diagram

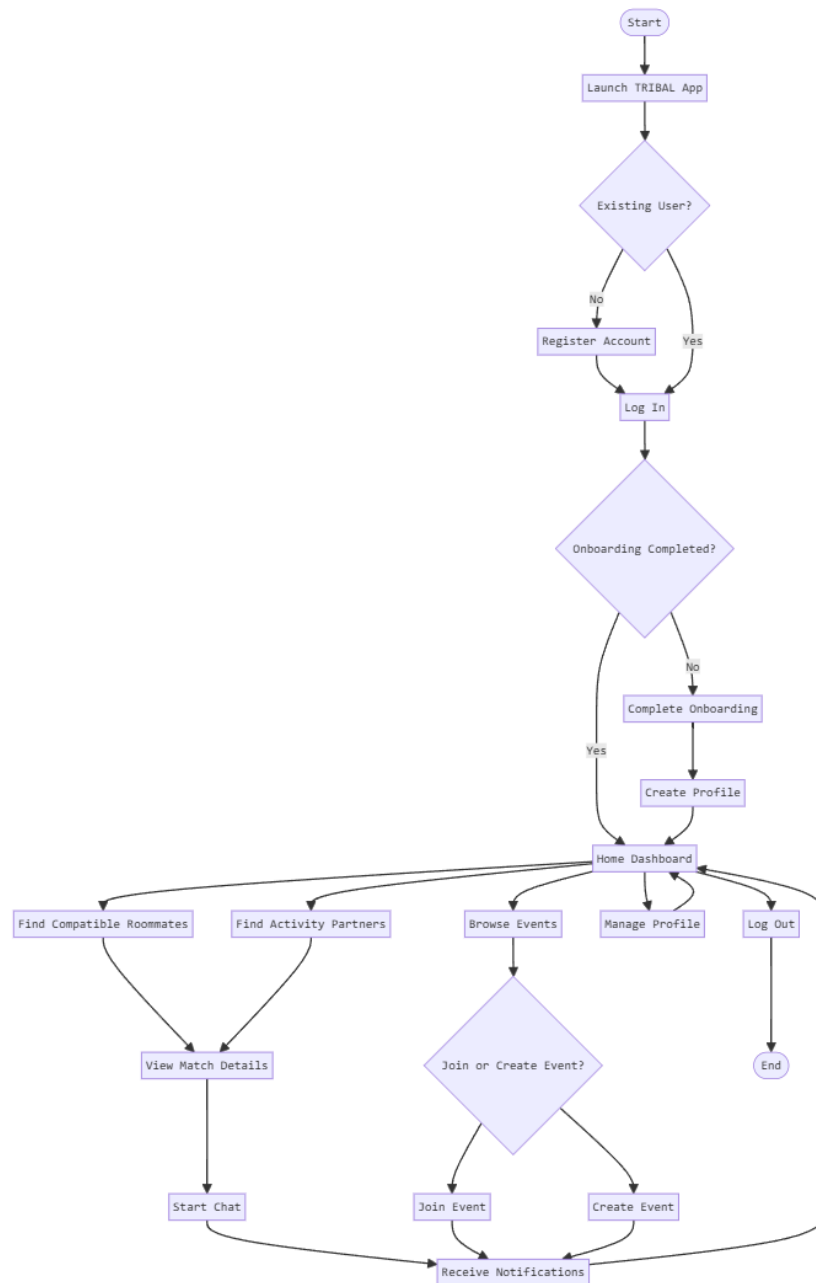


Figure 2.6: User Flow Diagram

Figure 2.6 shows how a user moves through the TRIBAL application from start to finish. The flow begins with launching the app. The first decision is whether the user is an existing user or not. If not, they register an account; if yes, they go straight to Log In.

After logging in, there is a second decision: whether onboarding has been completed. If not, the user is taken to Complete Onboarding, and then to Create Profile. If yes, they skip ahead. After the profile is created, the user reaches the Home Dashboard, from which five paths branch out: Find Compatible Roommates, Find Activity Partners, Browse Events, Manage Profile, and Log Out. The first two lead to View Match Details, which further leads to Start Chat. The Browse Events path leads to a decision diamond “Join or Create Event?”, which splits into Join Event and Create Event. Both event paths and the Start Chat path converge back to Receive Notifications. The Log Out path ends the session. This diagram helped us plan the navigation structure of the app early on so that all screens are connected in a logical way.

2.6 Tools Used

The following tools and technologies are being used in the development of TRIBAL:

Category	Tool / Technology
Frontend Framework	Flutter
Frontend Language	Dart
State Management	Provider
Navigation	GoRouter
Backend Framework	Django
Backend Language	Python
Database	PostgreSQL
Authentication	JWT
Real-Time Communication	Django Channels, WebSockets
Version Control	Git, GitHub
Code Editor	Visual Studio Code
Mobile Testing	Android Studio
API Testing	Postman
UI/UX Design	Figma

Table 2.3: Tools and Technologies Used

Flutter was chosen for the frontend because it allows the team to write one codebase that works on both Android and iOS. Django was chosen for the backend because it comes with many built-in features like an ORM and an admin panel, which makes development faster for a project with this scope. PostgreSQL was chosen as the database because it works well with Django and can reliably handle relational data like user profiles and onboarding responses.

Chapter 3 Epilogue

3.1 Tasks Completed

So far, the team has completed the following tasks:

- UI design for onboarding, login, signup, and profile setup screens in Flutter.
- Implementation of the three-slide onboarding screen with skip and next functionality.
- Animated loading screen shown during navigation between major flows.
- Django project setup with PostgreSQL as the database.
- JWT-based authentication APIs for login and signup.
- Database models for User and RoommateProfile.
- Design of the roommate compatibility scoring algorithm with weighted factors and deal breaker rules.
- Manual API testing using Postman.
- GitHub repository setup for version control and team collaboration.

3.2 Remaining Tasks

The following tasks are still remaining and will be completed in the upcoming sprints before the final defense:

- Implementation of the scoring logic inside the roommate module (models, scoring, services, views, serializers, urls, admin).
- Initial unit tests for the scoring function.
- Integration of the roommate matching API with the Flutter frontend, so that real compatibility scores are shown in the app instead of placeholder data.
- Implementation of the real-time chat system using Django Channels and WebSockets.
- Chat screens in Flutter for one-to-one conversations between matched users.
- Event creation and joining module, including event listing and filtering screens.
- Notification system for new matches and messages.
- Full integration testing between the Flutter frontend and the Django backend.

- Final round of bug fixing and performance testing.
- Preparation of the user manual and final project documentation.
- Deployment of the backend and preparation for the final presentation.

3.3 Updated Gantt Chart

The original time schedule has been adjusted slightly based on the actual pace of development so far. The updated schedule below shows the completed sprints and the plan for the remaining weeks leading up to the final defense.

Tasks	Weeks												
	2	4	6	8	10	12	14	16	18	20	22	24	
Requirement Analysis													
System Design													
Implementation													
Testing and Evaluation													
Deployment													
Monitoring													
Documentation													

The team is currently on Sprint 5. Based on the progress made so far, the team is confident that the remaining sprints can be completed within the original 16-week timeline, with some buffer time set aside before the final defense for bug fixing and documentation.

References and Bibliography

- [1] G. Liao, X. Huang, N. N. Xiong, and C. Wan, “An intelligent group event recommendation system in social networks,” *Jiangxi University of Finance and Economics*, 2016.
- [2] H. Hosseini, S. Narang, and S. Roy, “Strategyproof matching of roommates and rooms,” in *Proceedings of the 39th AAAI Conference on Artificial Intelligence (AAAI-25)*, 2025.
- [3] E. Molis and R. F. Veszteg, “Experimental results on the roommate problem,” *SSRN Electronic Journal*, 2023, Waseda University.
- [4] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner’s Approach*, 9th ed. McGraw-Hill, 2019.
- [5] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2017.
- [6] Flutter. “Flutter documentation.”<https://docs.flutter.dev>.
- [7] Django Software Foundation. “Django documentation.”<https://docs.djangoproject.com>.
- [8] Google. “Material design 3.”<https://m3.material.io>.

Screenshots

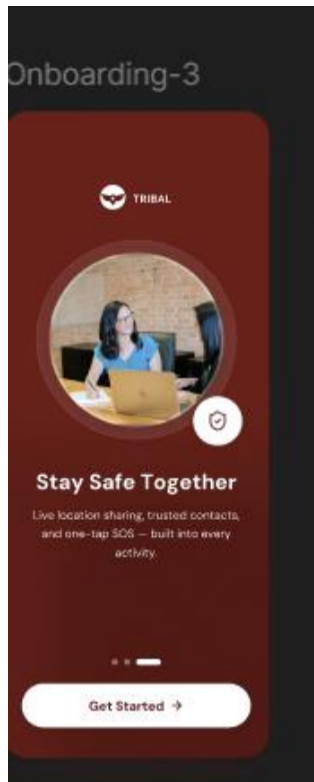


(a) Screenshot 1

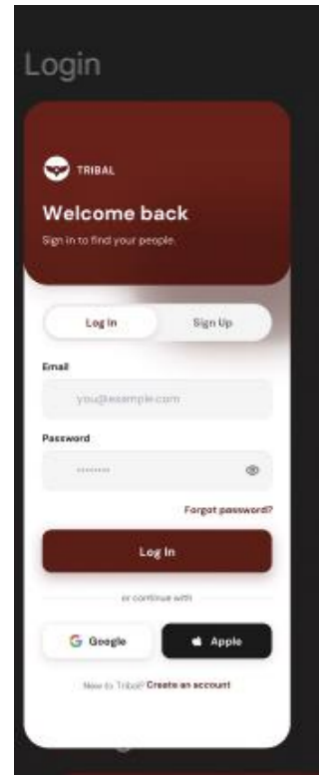


(b) Screenshot 2

Figure 1: TRIBAL Application Screenshots (Part 1)

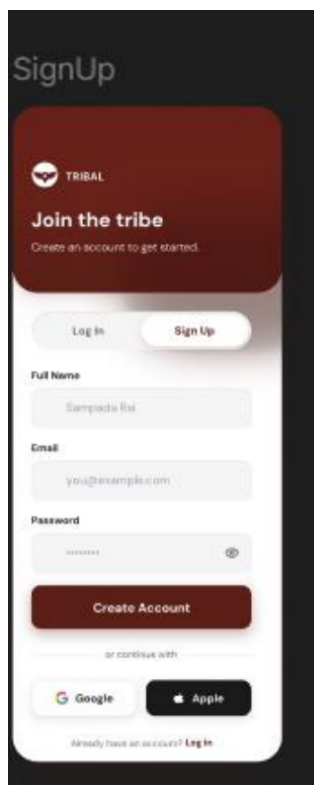


(a) Screenshot 3

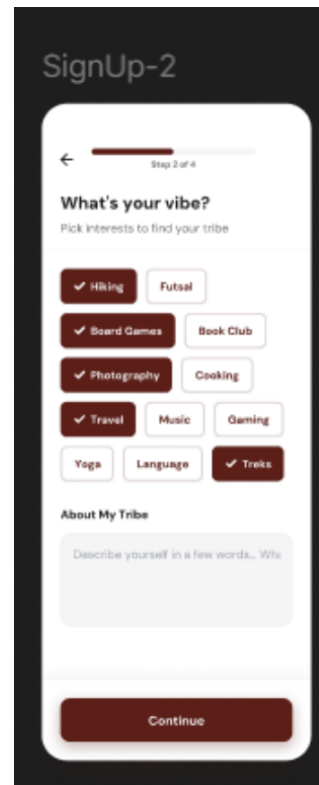


(b) Screenshot 4

Figure 2: TRIBAL Application Screenshots (Part 2)



(a) Screenshot 5



(b) Screenshot 6

Figure 3: TRIBAL Application Screenshots (Part 3)